

Experiment-5

Aim :

To create SQL views for department salary summary and employee hierarchy, test the updatability of views, and implement a recursive CTE to display reporting chains.

Description :

A SQL view is a virtual table based on the result of a SELECT query. Views are useful for presenting selected data, simplifying complex queries, and providing controlled access to database information.

In this experiment, two views are created. The first view, department_salary_summary, groups employees by department and displays the number of employees, total salary, and average salary. The second view, employee_hierarchy, displays employee details together with their manager information. The updatability of a simple employee view is then tested using an UPDATE statement.

A recursive Common Table Expression (CTE) is also implemented to display the complete reporting chain of employees. The recursive CTE starts from top-level employees who do not report to another employee and repeatedly joins employees with their managers.

The following SQL uses MySQL 8.0 or later syntax because recursive CTEs require support for WITH RECURSIVE.

Source Code :

-- Create database objects for the experiment

```
CREATE TABLE department ( department_id INT
    PRIMARY KEY, department_name
    VARCHAR(50) NOT NULL
);
```

```
CREATE TABLE employee ( employee_id INT
    PRIMARY KEY, employee_name
    VARCHAR(100) NOT NULL,
    department_id INT,
    manager_id INT NULL,
    salary
    DECIMAL(10,2),
    FOREIGN KEY (department_id) REFERENCES department(department_id),
    FOREIGN KEY (manager_id) REFERENCES employee(employee_id)
);
```

-- Insert sample departments

```
INSERT INTO department VALUES
(1, 'IT'),
(2, 'HR'),
```

(3, 'Finance');

-- Insert sample employees

```
INSERT INTO employee VALUES
(101, 'Amit', 1, NULL, 90000),
(102, 'Neha', 1, 101, 65000),
(103, 'Rahul', 1, 101, 60000),
(104, 'Priya', 2, NULL, 80000),
(105, 'Karan', 2, 104, 55000),
(106, 'Riya', 3, NULL, 85000),
(107, 'Arjun', 3, 106, 50000);
```

-- 1. View for department salary summary

```
CREATE VIEW department_salary_summary AS
SELECT
    d.department_id,
    d.department_name,
    COUNT(e.employee_id) AS employee_count,
    SUM(e.salary) AS total_salary,
    AVG(e.salary) AS average_salary
FROM department d
LEFT JOIN employee e
    ON d.department_id = e.department_id
GROUP BY d.department_id, d.department_name;

SELECT * FROM department_salary_summary;
```

-- 2. View for employee hierarchy

```
CREATE VIEW employee_hierarchy AS
SELECT
    e.employee_id,
    e.employee_name,
    e.department_id,
    e.manager_id,
    m.employee_name AS manager_name,
    e.salary
FROM employee e
LEFT JOIN employee m
    ON e.manager_id = m.employee_id;

SELECT * FROM employee_hierarchy;
```

-- 3. Test view updatability using a simple single-table view

```
CREATE VIEW employee_basic AS
SELECT employee_id, employee_name, salary
FROM employee;
```

-- Update through the view

```
UPDATE employee_basic
SET salary = salary + 5000
WHERE employee_id = 102;
```

```
SELECT * FROM employee_basic
WHERE employee_id = 102;
```

-- 4. Recursive CTE to display reporting chains

```
WITH RECURSIVE reporting_chain AS (
  -- Anchor member: top-level employees
  SELECT
    employee_id,
    employee_name,
    manager_id,
    0 AS level,
    CAST(employee_name AS CHAR(1000)) AS reporting_path
  FROM employee
  WHERE manager_id IS NULL

  UNION ALL

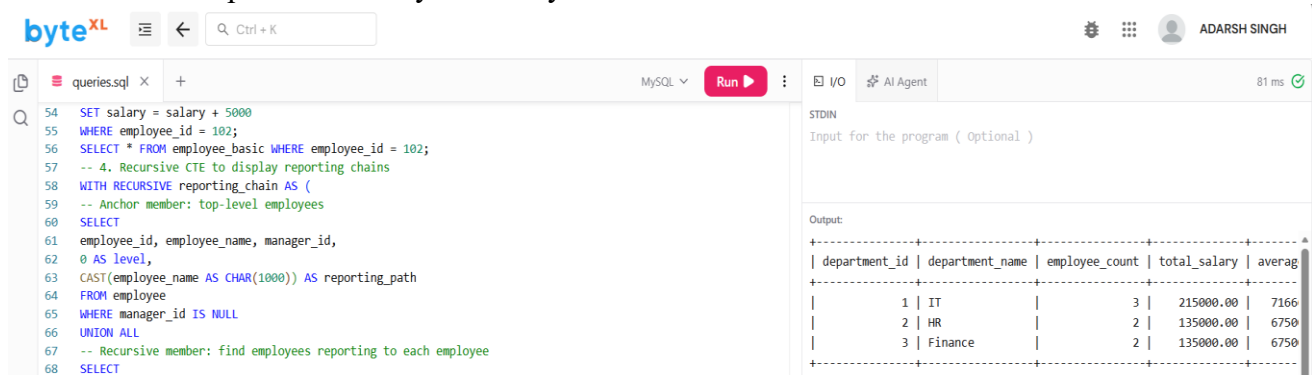
  -- Recursive member: find employees reporting to each employee
  SELECT
    e.employee_id,
    e.employee_name,
    e.manager_id,
    rc.level + 1,
    CONCAT(rc.reporting_path, ' -> ', e.employee_name)
  FROM employee e
  INNER JOIN reporting_chain rc
    ON e.manager_id = rc.employee_id
)
SELECT
  employee_id,
  employee_name,
  manager_id,
  level,
  reporting_path
FROM reporting_chain
```

ORDER BY reporting_path;

Output :

Run the SQL statements and capture screenshots showing the department salary summary, employee hierarchy, successful update through the simple view, and recursive reporting chain.

Screenshot 1: Department Salary Summary



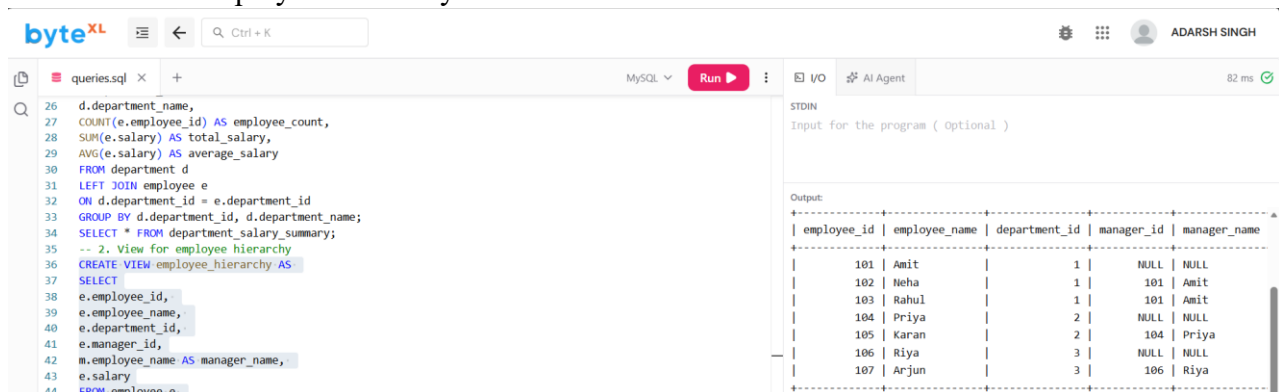
The screenshot shows a SQL IDE with a query window and an output window. The query window contains the following SQL code:

```
54 SET salary = salary + 5000
55 WHERE employee_id = 102;
56 SELECT * FROM employee_basic WHERE employee_id = 102;
57 -- 4. Recursive CTE to display reporting chains
58 WITH RECURSIVE reporting_chain AS (
59 -- Anchor member: top-level employees
60 SELECT
61 employee_id, employee_name, manager_id,
62 0 AS level,
63 CAST(employee_name AS CHAR(1000)) AS reporting_path
64 FROM employee
65 WHERE manager_id IS NULL
66 UNION ALL
67 -- Recursive member: find employees reporting to each employee
68 SELECT
```

The output window displays the following table:

department_id	department_name	employee_count	total_salary	average
1	IT	3	215000.00	7166
2	HR	2	135000.00	6750
3	Finance	2	135000.00	6750

Screenshot 2: Employee Hierarchy



The screenshot shows a SQL IDE with a query window and an output window. The query window contains the following SQL code:

```
26 d.department_name,
27 COUNT(e.employee_id) AS employee_count,
28 SUM(e.salary) AS total_salary,
29 AVG(e.salary) AS average_salary
30 FROM department d
31 LEFT JOIN employee e
32 ON d.department_id = e.department_id
33 GROUP BY d.department_id, d.department_name;
34 SELECT * FROM department_salary_summary;
35 -- 2. View for employee hierarchy
36 CREATE VIEW employee_hierarchy AS
37 SELECT
38 e.employee_id,
39 e.employee_name,
40 e.department_id,
41 e.manager_id,
42 m.employee_name AS manager_name,
43 e.salary
44 FROM employee e
```

The output window displays the following table:

employee_id	employee_name	department_id	manager_id	manager_name
101	Amit	1	NULL	NULL
102	Neha	1	101	Amit
103	Rahul	1	101	Amit
104	Priya	2	NULL	NULL
105	Karan	2	104	Priya
106	Riya	3	NULL	NULL
107	Arjun	3	106	Riya

Screenshot 3: View Updatability Test

employee_id	employee_name	salary
102	Neha	70000.00

Screenshot 4: Recursive CTE Reporting Chain

byteXL

queries.sql x MySQL Run

ADARSH SINGH

```
38 WITH RECURSIVE reporting_chain AS (  
40     SELECT  
43         manager_id,  
44         0 AS level,  
45         CAST(employee_name AS CHAR(100)) AS reporting_path  
46     FROM employee  
47     WHERE manager_id IS NULL  
48  
49     UNION ALL  
50  
51     -- Recursive member: find employees reporting to  
52     SELECT  
53         e.employee_id,  
54         e.employee_name,  
55         e.manager_id,  
56         rc.level + 1,  
57         CONCAT(rc.reporting_path, ' -> ', e.employee_name) AS reporting_path  
58     FROM employee e  
59     INNER JOIN reporting_chain rc  
60         ON e.manager_id = rc.employee_id  
61 )  
62 SELECT employee_id, employee_name, manager_id, level, reporting_path
```

STDIN

Input for the program (Optional)

Output:

employee_id	employee_name	manager_id	level	reporting_path
101	Amit	NULL	0	Amit
102	Neha	101	1	Amit -> Neha
103	Rahul	101	1	Amit -> Rahul
104	Priya	NULL	0	Priya
105	Karan	104	1	Priya -> Karan
106	Riya	NULL	0	Riya
107	Arjun	106	1	Riya -> Arjun

Ready

Feedback

New History Save